spring aop讲义

- 我们如果想开启aop，得在xml文件中配置相应的标签，而这个标签都会有响应的解析器解析aop:aspectj-autoproxy/

```
pplication context not configured for this file                    Configure appli
<?xml version="1.0" encoding="UTF-8"?>

<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xmlns:p="http://www.springframework.org/schema/p"
       xmlns:aop="http://www.springframework.org/schema/aop"
       xsi:schemaLocation="
           http://www.springframework.org/schema/beans
           http://www.springframework.org/schema/beans/spring-beans-3.2.xsd">
    <aop:aspectj-autoproxy/>
```

- 寻找 aop:aspectj-autoproxy/ 注解对应的解析器,会发现在AopNamespaceHandler类中有这个注解，主要是为了去得到AspectJAutoProxyBeanDefinitionParser解析器

```
public class AopNamespaceHandler extends NamespaceHandlerSupport {
    @Override
    public void init() {
        registerBeanDefinitionParser("config", new ConfigBeanDefinitionParser());
        registerBeanDefinitionParser("aspectj-autoproxy", new
AspectJAutoProxyBeanDefinitionParser());// 就是该段代码
        registerBeanDefinitionDecorator("scoped-proxy", new
ScopedProxyBeanDefinitionDecorator());

        registerBeanDefinitionParser("spring-configured", new
SpringConfiguredBeanDefinitionParser());
    }
}
```

- 了解AspectJAutoProxyBeanDefinitionParser对应的行为，主要是为了得到一个AnnotationAwareAspectJAutoProxyCreator类，实现了BeanPostProcessor接口，实现类为AbstractAutoProxyCreator，这个类我们在后置通知的时候有看到，根据来得到一个代理对象

```
class AspectJAutoProxyBeanDefinitionParser implements BeanDefinitionParser {

    @Override
    public BeanDefinition parse(Element element, ParserContext parserContext) {
        // 1.注册proxy creator

AopNamespaceUtils.registerAspectJAnnotationAutoProxyCreatorIfNecessary(parserContext,
 element);
        extendBeanDefinition(element, parserContext);
        return null;
    }
    ...
```

```
    public static void registerAspectJAnnotationAutoProxyCreatorIfNecessary(
            ParserContext parserContext, Element sourceElement) {
        // 注册行为主要内容
        BeanDefinition beanDefinition =
AopConfigUtils.registerAspectJAnnotationAutoProxyCreatorIfNecessary(
                parserContext.getRegistry(),
parserContext.extractSource(sourceElement));
        useClassProxyingIfNecessary(parserContext.getRegistry(), sourceElement);
        registerComponentIfNecessary(beanDefinition, parserContext);
    }

    public static BeanDefinition
registerAspectJAnnotationAutoProxyCreatorIfNecessary(BeanDefinitionRegistry registry,
Object source) {
        // 主要就是为了注册AnnotationAwareAspectJAutoProxyCreator类
        return
registerOrEscalateApcAsRequired(AnnotationAwareAspectJAutoProxyCreator.class,
registry, source);
    }

    // 注册类相关代码
    private static BeanDefinition registerOrEscalateApcAsRequired(Class<?> cls,
BeanDefinitionRegistry registry, Object source) {
        Assert.notNull(registry, "BeanDefinitionRegistry must not be null");
        if (registry.containsBeanDefinition(AUTO_PROXY_CREATOR_BEAN_NAME)) {
            BeanDefinition apcDefinition =
registry.getBeanDefinition(AUTO_PROXY_CREATOR_BEAN_NAME);
            if (!cls.getName().equals(apcDefinition.getBeanClassName())) {
                int currentPriority =
findPriorityForClass(apcDefinition.getBeanClassName());
                int requiredPriority = findPriorityForClass(cls);
                if (currentPriority < requiredPriority) {
                    apcDefinition.setBeanClassName(cls.getName());
                }
            }
            return null;
        }

        // 类似于我们在使用BeanFactory.getBean()时候的操作，生成一个RootBeanDefinition，然后
放入map中
        RootBeanDefinition beanDefinition = new RootBeanDefinition(cls);
        beanDefinition.setSource(source);
        beanDefinition.getPropertyValues().add("order", Ordered.HIGHEST_PRECEDENCE);
        beanDefinition.setRole(BeanDefinition.ROLE_INFRASTRUCTURE);
        registry.registerBeanDefinition(AUTO_PROXY_CREATOR_BEAN_NAME,
beanDefinition);
        return beanDefinition;
```

- 总结：通过以上的代码分析，可知，AspectJAutoProxyBeanDefinitionParser主要的功能就是将AnnotationAwareAspectJAutoProxyCreator注册到Spring容器中，把bean交给Spring去托管。

  AnnotationAwareAspectJAutoProxyCreator的功能我们大胆猜测一下：应该也就是生成对象的代理类的相关功能，这个我们接下来再看。

- 问题：那么问题来了，我们最开始的类AopNamespaceHandler.init()方法是在什么时候被调用的呢？什么时候生效的？这个决定了我们注册到Spring的AnnotationAwareAspectJAutoProxyCreator的生效时间？

```
AbstractXmlApplicationContext.java   AbstractRefreshableApplicationContext.java   AbstractApplicationContext.java

public void refresh() throws BeansException, IllegalStateException {
    // 用startupShutdownMonitor来加一把锁：用于“刷新”和“销毁”的同步监视器
    synchronized (this.startupShutdownMonitor) {

        // Prepare this context for refreshing.
        //1、调用容器准备刷新的方法，获取容器的当时时间，同时给容器设置同步标识，已启动标志
        prepareRefresh();

        // Tell the subclass to refresh the internal bean factory.
        //2、告诉子类启动refreshBeanFactory()方法，Bean定义资源文件的载入从
        //子类的refreshBeanFactory()方法启动-------加载，解析xml文件------
        ConfigurableListableBeanFactory beanFactory = obtainFreshBeanFactory();

        // Prepare the bean factory for use in this context.
        //3、为BeanFactory配置容器特性，例如类加载器、事件处理器等
```

- AbstractApplicationContext中的refreshBeanFactory()方法

```
AbstractXmlApplicationContext.java   AbstractRefreshableApplicationContext.java   AbstractApplicationContext.java

/**
 * Tell the subclass to refresh the internal bean factory.
 * @return the fresh BeanFactory instance
 * @see #refreshBeanFactory()
 * @see #getBeanFactory()
 */
protected ConfigurableListableBeanFactory obtainFreshBeanFactory() {
    // 父类定义了抽象的refreshBeanFactory()方法，具体实现调用子类容器的refreshBeanFactory()方法-----往下
    this.refreshBeanFactory();

    ConfigurableListableBeanFactory beanFactory = getBeanFactory();
    if (logger.isDebugEnabled()) {
        logger.debug("Bean factory for " + getDisplayName() + ": " + beanFactory);
    }
    return beanFactory;
}
```

- AbstractRefreshableApplicationContext中的refreshBeanFactory方法

```java
protected final void refreshBeanFactory() throws BeansException {
    //如果已经有容器，销毁容器中的bean，关闭容器
    if (hasBeanFactory()) {
        destroyBeans();
        closeBeanFactory();
    }
    try {
        //创建IOC容器
        DefaultListableBeanFactory beanFactory = createBeanFactory();
        beanFactory.setSerializationId(getId());
        //对IOC容器进行定制化，如设置启动参数，开启注解的自动装配等
        customizeBeanFactory(beanFactory);
        //调用载入Bean定义的方法，主要这里又使用了一个委派模式，在当前类中只定义了抽象的loadBeanDefinitions方法，
        loadBeanDefinitions(beanFactory);
        synchronized (this.beanFactoryMonitor) {
            this.beanFactory = beanFactory;
        }
    }
```

- AbstractXmlApplicationContext中的loadBeanDefinitions方法

```java
 * @see org.springframework.beans.factory.xml.XmlBeanDefinitionReader
 * @see #initBeanDefinitionReader
 * @see #loadBeanDefinitions
 */
//实现父类抽象的载入Bean定义方法
@Override
protected void loadBeanDefinitions(DefaultListableBeanFactory beanFactory) throws BeansException, IOException {
    // Create a new XmlBeanDefinitionReader for the given BeanFactory.
    //创建XmlBeanDefinitionReader，即创建Bean读取器，并通过回调设置到容器中去，容器使用该读取器读取Bean定义资源
    XmlBeanDefinitionReader beanDefinitionReader = new XmlBeanDefinitionReader(beanFactory);

    // Configure the bean definition reader with this context's
    // resource loading environment.
    //为Bean读取器设置Spring资源加载器，AbstractXmlApplicationContext的
    //祖先父类AbstractApplicationContext继承DefaultResourceLoader，因此，容器本身也是一个资源加载器
    beanDefinitionReader.setEnvironment(this.getEnvironment());
    beanDefinitionReader.setResourceLoader(this);
    //为Bean读取器设置SAX xml解析器
    beanDefinitionReader.setEntityResolver(new ResourceEntityResolver( resourceLoader: this));

    // Allow a subclass to provide custom initialization of the reader,
    // then proceed with actually loading the bean definitions.
    //当Bean读取器读取Bean定义的Xml资源文件时，启用Xml的校验机制
    initBeanDefinitionReader(beanDefinitionReader);
    //Bean读取器真正实现加载的方法----------往下走----------
    loadBeanDefinitions(beanDefinitionReader);
}
```

- loadBeanDefinitions方法

```java
 * @see #getResourceBeanNameGenerator
 */
//Xml Bean读取器加载Bean定义资源
protected void loadBeanDefinitions(XmlBeanDefinitionReader reader) throws BeansException, IOException {
    //获取Bean定义资源的定位
    Resource[] configResources = getConfigResources();
    if (configResources != null) {
        //Xml Bean读取器调用其父类AbstractBeanDefinitionReader读取定位
        //的Bean定义资源-----------往下走----------
        reader.loadBeanDefinitions(configResources);
    }
    //如果子类中获取的Bean定义资源定位为空，则获取FileSystemXmlApplicationContext构造方法中setConfigLocations方法设置
    String[] configLocations = getConfigLocations();
    if (configLocations != null) {
        //Xml Bean读取器调用其父类AbstractBeanDefinitionReader读取定位
        //的Bean定义资源
        reader.loadBeanDefinitions(configLocations);
    }
}
```

- XmlBeanDefinitionReader.doLoadBeanDefinitions方法

```java
//这里是载入XML形式Bean定义资源文件方法
public int loadBeanDefinitions(EncodedResource encodedResource) throws BeanDefinitionStoreException {
    Assert.notNull(encodedResource, message: "EncodedResource must not be null");
    if (logger.isInfoEnabled()) {
        logger.info("Loading XML bean definitions from " + encodedResource.getResource());
    }

    Set<EncodedResource> currentResources = this.resourcesCurrentlyBeingLoaded.get();
    if (currentResources == null) {
        currentResources = new HashSet<>( initialCapacity: 4);
        this.resourcesCurrentlyBeingLoaded.set(currentResources);
    }
    if (!currentResources.add(encodedResource)) {
        throw new BeanDefinitionStoreException(
                "Detected cyclic loading of " + encodedResource + " - check your import definitions!");
    }
    try {
        //将资源文件转为InputStream的IO流
        InputStream inputStream = encodedResource.getResource().getInputStream();
        try {
            //从InputStream中得到XML的解析源
            InputSource inputSource = new InputSource(inputStream);
            if (encodedResource.getEncoding() != null) {
                inputSource.setEncoding(encodedResource.getEncoding());
            }
            //这里是具体的读取过程------往下走-----------
            return doLoadBeanDefinitions(inputSource, encodedResource.getResource());
        }
        finally {
            //关闭从Resource中得到的IO流
            inputStream.close();
```

- XmlBeanDefinitionReader.registerBeanDefinitions方法

```java
XmlBeanDefinitionReader.java  XmlBeanFactory.java  ClassPathXmlApplicationContext.java  AbstractApplicationContext.java  AbstractRefreshableApplicationContext.ja
     * {@code registerBeanDefinitions} on it.
     * @param doc the DOM document
     * @param resource the resource descriptor (for context information)
     * @return the number of bean definitions found
     * @throws BeanDefinitionStoreException in case of parsing errors
     * @see #loadBeanDefinitions
     * @see #setDocumentReaderClass
     * @see BeanDefinitionDocumentReader#registerBeanDefinitions
     */
    //按照Spring的Bean语义要求将Bean定义资源解析并转换为容器内部数据结构
    public int registerBeanDefinitions(Document doc, Resource resource) throws BeanDefinitionStoreException {
        //得到BeanDefinitionDocumentReader来对xml格式的BeanDefinition解析
        BeanDefinitionDocumentReader documentReader = createBeanDefinitionDocumentReader();
        //获得容器中注册的Bean数量
        int countBefore = getRegistry().getBeanDefinitionCount();
        //解析过程入口，这里使用了委派模式，BeanDefinitionDocumentReader只是个接口，
        //具体的解析实现过程有实现类DefaultBeanDefinitionDocumentReader完成----------往下走----------
        documentReader.registerBeanDefinitions(doc, createReaderContext(resource));
        //统计解析的Bean数量
        return getRegistry().getBeanDefinitionCount() - countBefore;
    }

    /**
```

- DefaultBeanDefinitionDocumentReader类中的registerBeanDefinitions方法

```java
DefaultBeanDefinitionDocumentReader.java  XmlBeanDefinitionReader.java  BeanDefinitionDocumentReader.java  XmlBeanFactory.java  Cl


    /**
     * This implementation parses bean definitions according to the "spring-beans" XSD
     * (or DTD, historically).
     * <p>Opens a DOM Document; then initializes the default settings
     * specified at the {@code <beans/>} level; then parses the contained bean definitions.
     */
    //根据Spring DTD对Bean的定义规则解析Bean定义Document对象
    @Override
    public void registerBeanDefinitions(Document doc, XmlReaderContext readerContext) {
        //获得XML描述符
        this.readerContext = readerContext;
        logger.debug("Loading bean definitions");
        //获得Document的根元素
        Element root = doc.getDocumentElement();
        doRegisterBeanDefinitions(root);
    }

    /**
```

- DefaultBeanDefinitionDocumentReader类中的doRegisterBeanDefinitions方法

```java
DefaultBeanDefinitionDocumentReader.java  XmlBeanDefinitionReader.java  BeanDefinitionDocumentReader.java  XmlBeanFactory.java  ClassP

    */
protected void doRegisterBeanDefinitions(Element root) {

    //具体的解析过程由BeanDefinitionParserDelegate实现,
    //BeanDefinitionParserDelegate中定义了Spring Bean定义XML文件的各种元素
    BeanDefinitionParserDelegate parent = this.delegate;
    this.delegate = createDelegate(getReaderContext(), root, parent);

    if (this.delegate.isDefaultNamespace(root)) {
        String profileSpec = root.getAttribute(PROFILE_ATTRIBUTE);
        if (StringUtils.hasText(profileSpec)) {
            String[] specifiedProfiles = StringUtils.tokenizeToStringArray(
                    profileSpec, BeanDefinitionParserDelegate.MULTI_VALUE_ATTRIBUTE_DELIMITERS);
            if (!getReaderContext().getEnvironment().acceptsProfiles(specifiedProfiles)) {
                if (logger.isInfoEnabled()) {
                    logger.info("Skipped XML bean definition file due to specified profiles [" + profileSpec +
                            "] not matching: " + getReaderContext().getResource());
                }
                return;
            }
        }
    }

    //在解析Bean定义之前，进行自定义的解析，增强解析过程的可扩展性
    preProcessXml(root);
    //从Document的根元素开始进行Bean定义的Document对象----------往下走----------
    parseBeanDefinitions(root, this.delegate);
    //在解析Bean定义之后，进行自定义的解析，增加解析过程的可扩展性
    postProcessXml(root);
```

- DefaultBeanDefinitionDocumentReader类中的parseBeanDefinitions

```java
DefaultBeanDefinitionDocumentReader.java  XmlBeanDefinitionReader.java  BeanDefinitionDocumentReader.java  XmlBeanFactory.java

     * "import", "alias", "bean".
     * @param root the DOM root element of the document
     */
    //使用Spring的Bean规则从Document的根元素开始进行Bean定义的Document对象
    protected void parseBeanDefinitions(Element root, BeanDefinitionParserDelegate delegate) {
        //Bean定义的Document对象使用了Spring默认的XML命名空间
        if (delegate.isDefaultNamespace(root)) {
            //获取Bean定义的Document对象根元素的所有子节点
            NodeList nl = root.getChildNodes();
            for (int i = 0; i < nl.getLength(); i++) {
                Node node = nl.item(i);
                //获得Document节点是XML元素节点
                if (node instanceof Element) {
                    Element ele = (Element) node;
                    //Bean定义的Document的元素节点使用的是Spring默认的XML命名空间
                    if (delegate.isDefaultNamespace(ele)) {
                        //使用Spring的Bean规则解析元素节点--------往下走---------------
                        parseDefaultElement(ele, delegate);
                    }
                    else {
                        //没有使用Spring默认的XML命名空间，则使用用户自定义的解析规则解析元素节点
                        delegate.parseCustomElement(ele);
                    }
                }
            }
        }
        else {
            //Document的根节点没有使用Spring默认的命名空间，则使用用户自定义的
            //解析规则解析Document根节点
            delegate.parseCustomElement(root);
        }
```

- BeanDefinitionParserDelegate类中的parseCustomElement

```java
DefaultBeanDefinitionDocumentReader.java  BeanDefinitionParserDelegate.java  XmlBeanDefinitionReader.java  BeanDefinitionDocumentReader.java  ClassF

    }
    return TRUE_VALUE.equals(value);
}

@Nullable
public BeanDefinition parseCustomElement(Element ele) {
    return parseCustomElement(ele, containingBd: null);
}

@Nullable
public BeanDefinition parseCustomElement(Element ele, @Nullable BeanDefinition containingBd) {
    String namespaceUri = getNamespaceURI(ele);
    if (namespaceUri == null) {
        return null;
    }
    NamespaceHandler handler = this.readerContext.getNamespaceHandlerResolver().resolve(namespaceUri);
    if (handler == null) {
        error( message: "Unable to locate Spring NamespaceHandler for XML schema namespace [" + namespaceUri + "]", ele);
        return null;
    }
    return handler.parse(ele, new ParserContext(this.readerContext, delegate: this, containingBd));
}
```

- DefaultNamespaceHandlerResolver类中的resolve方法

DefaultBeanDefinitionDocumentReader.java | BeanDefinitionParserDelegate.java | NamespaceHandlerResolver.java | DefaultNamespaceHandlerResolver.java | XmlBea

```java
113     * @return the located {@link NamespaceHandler}, or {@code null} if none found
114     */
115    @Override
116    @Nullable
117    public NamespaceHandler resolve(String namespaceUri) {
118        Map<String, Object> handlerMappings = getHandlerMappings();
119        Object handlerOrClassName = handlerMappings.get(namespaceUri);
120        if (handlerOrClassName == null) {
121            return null;
122        }
123        else if (handlerOrClassName instanceof NamespaceHandler) {
124            return (NamespaceHandler) handlerOrClassName;
125        }
126        else {
127            String className = (String) handlerOrClassName;
128            try {
129                Class<?> handlerClass = ClassUtils.forName(className, this.classLoader);
130                if (!NamespaceHandler.class.isAssignableFrom(handlerClass)) {
131                    throw new FatalBeanException("Class [" + className + "] for namespace [" + namespaceUri +
132                            "] does not implement the [" + NamespaceHandler.class.getName() + "] interface");
133                }
134                NamespaceHandler namespaceHandler = (NamespaceHandler) BeanUtils.instantiateClass(handlerClass);
135                namespaceHandler.init();
136                handlerMappings.put(namespaceUri, namespaceHandler);
137                return namespaceHandler;
138            }
139            catch (ClassNotFoundException ex) {
140                throw new FatalBeanException("NamespaceHandler class [" + className + "] for namespace [" +
141                        namespaceUri + "] not found", ex);
142            }
143            catch (LinkageError err) {
```

- AopNamespaceHandler.init()方法

```java
public class AopNamespaceHandler extends NamespaceHandlerSupport {

    /**
     * Register the {@link BeanDefinitionParser BeanDefinitionParsers} for the
     * '{@code config}', '{@code spring-configured}', '{@code aspectj-autoproxy}'
     * and '{@code scoped-proxy}' tags.
     */
    @Override
    public void init() {
        // In 2.0 XSD as well as in 2.1 XSD.
        registerBeanDefinitionParser( elementName: "config", new ConfigBeanDefinitionParser());
        registerBeanDefinitionParser( elementName: "aspectj-autoproxy", new AspectJAutoProxyBeanDefinitionParser());
        registerBeanDefinitionDecorator( elementName: "scoped-proxy", new ScopedProxyBeanDefinitionDecorator());

        // Only in 2.0 XSD: moved to context namespace as of 2.1
        registerBeanDefinitionParser( elementName: "spring-configured", new SpringConfiguredBeanDefinitionParser());
    }

}
```

- AspectJAutoProxyBeanDefinitionParser这个类中的parse方法，主要是为了得到AnnotationAwareAspectJAutoProxyCreator类

```
ndlerResolver.java × DefaultNamespaceHandlerResolver.java × NamespaceHandler.java × AopNamespaceHandler.java × AspectJAutoProxyBeanDefinitionParser.java × AopNamespaceUtils.
 * @author Rob Harrop
 * @author Juergen Hoeller
 * @since 2.0
 */
class AspectJAutoProxyBeanDefinitionParser implements BeanDefinitionParser {

    @Override
    @Nullable
    public BeanDefinition parse(Element element, ParserContext parserContext) {
        AopNamespaceUtils.registerAspectJAnnotationAutoProxyCreatorIfNecessary(parserContext, element);
        extendBeanDefinition(element, parserContext);
        return null;
    }

    private void extendBeanDefinition(Element element, ParserContext parserContext) {
        BeanDefinition beanDef =
                parserContext.getRegistry().getBeanDefinition(AopConfigUtils.AUTO_PROXY_CREATOR_BEAN_NAME);
        if (element.hasChildNodes()) {
            addIncludePatterns(element, parserContext, beanDef);
        }
    }
```

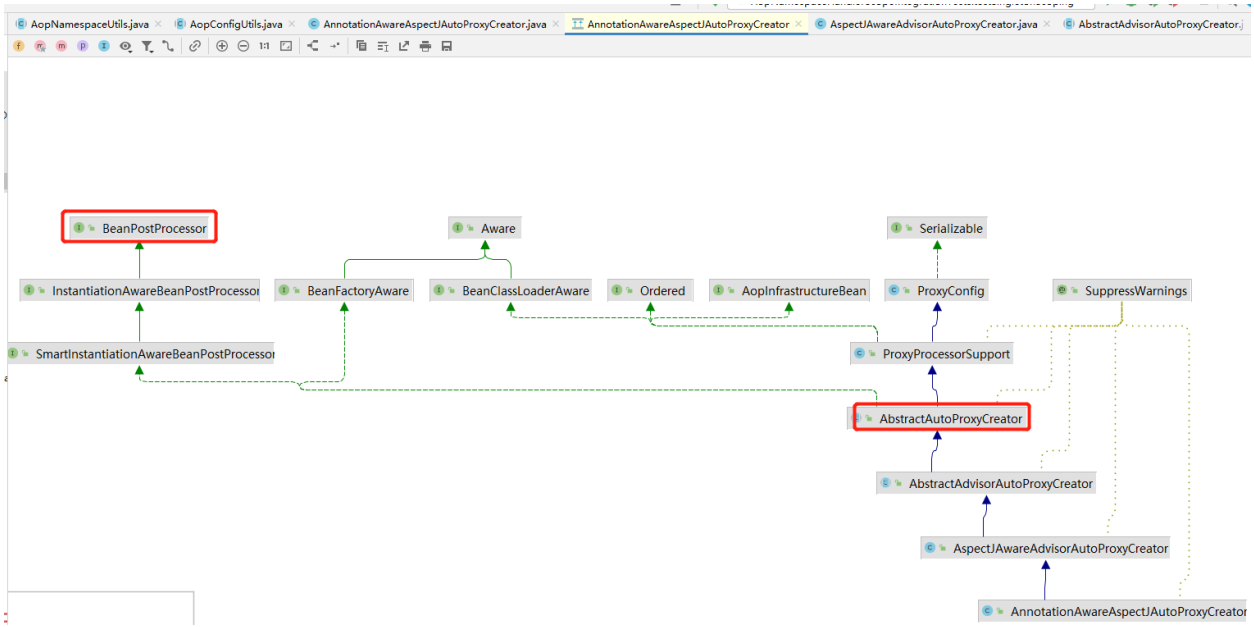
- AopNamespaceUtils.registerAspectJAnnotationAutoProxyCreatorIfNecessary方法

```
    public static void registerAspectJAnnotationAutoProxyCreatorIfNecessary(
            ParserContext parserContext, Element sourceElement) {

        BeanDefinition beanDefinition = AopConfigUtils.registerAspectJAnnotationAutoProxyCreatorIfNecessary(
                parserContext.getRegistry(), parserContext.extractSource(sourceElement));
        useClassProxyingIfNecessary(parserContext.getRegistry(), sourceElement);
        registerComponentIfNecessary(beanDefinition, parserContext);
    }
```

- 生成AnnotationAwareAspectJAutoProxyCreator类

```
@Nullable
public static BeanDefinition registerAspectJAnnotationAutoProxyCreatorIfNecessary(BeanDefinitionRegistry registry,
        @Nullable Object source) {

    return registerOrEscalateApcAsRequired(AnnotationAwareAspectJAutoProxyCreator.class, registry, source);
}
```

- 分析AnnotationAwareAspectJAutoProxyCreator主要行为， 通过查看AnnotationAwareAspectJAutoProxyCreator的类层次结构，可知，其实现了BeanPostProcessor接口，实现类为AbstractAutoProxyCreator



- AbstractAutoProxyCreator主要方法

```
@Override
public Object postProcessBeforeInitialization(Object bean, String beanName) {
    return bean;
}

// 主要看这个方法，在bean初始化之后对生产出的bean进行包装
@Override
public Object postProcessAfterInitialization(Object bean, String beanName) throws
BeansException {
    if (bean != null) {
        // 首先根据beanName拼接一个key
        Object cacheKey = getCacheKey(bean.getClass(), beanName);
        // 判断曾经是否生成过，这个地方如果存在，则说明当初从三级缓存中拿的时候，那个时候已经生成了
代理对象
        if (!this.earlyProxyReferences.contains(cacheKey)) {
            // 进入该方法判断是否生成代理对象
            return wrapIfNecessary(bean, beanName, cacheKey);
        }
    }
    return bean;
}

// wrapIfNecessary
protected Object wrapIfNecessary(Object bean, String beanName, Object cacheKey) {
    if (beanName != null && this.targetSourcedBeans.contains(beanName)) {
        return bean;
    }
    if (Boolean.FALSE.equals(this.advisedBeans.get(cacheKey))) {
        return bean;
    }
    if (isInfrastructureClass(bean.getClass()) || shouldSkip(bean.getClass(),
beanName)) {

        this.advisedBeans.put(cacheKey, Boolean.FALSE);
```

```
        return bean;
    }

    // Create proxy if we have advice.
    // 意思就是如果该类有advice则创建proxy,
    Object[] specificInterceptors = getAdvicesAndAdvisorsForBean(bean.getClass(),
beanName, null);
    if (specificInterceptors != DO_NOT_PROXY) {
        this.advisedBeans.put(cacheKey, Boolean.TRUE);
        // 1.通过方法名也能简单猜测到，这个方法就是把bean包装为proxy的主要方法,
        Object proxy = createProxy(
            bean.getClass(), beanName, specificInterceptors, new
SingletonTargetSource(bean));
        this.proxyTypes.put(cacheKey, proxy.getClass());

        // 2.返回该proxy代替原来的bean
        return proxy;
    }

    this.advisedBeans.put(cacheKey, Boolean.FALSE);
    return bean;
}
```

- 分析getAdvicesAndAdvisorsForBean方法，到底是如何得到切面的链的

```
protected Object[] getAdvicesAndAdvisorsForBean(
        Class<?> beanClass, String beanName, @Nullable TargetSource targetSource) {
    // 得到所有的切面的链的
    List<Advisor> advisors = findEligibleAdvisors(beanClass, beanName);
    if (advisors.isEmpty()) {
        return DO_NOT_PROXY;
    }
    return advisors.toArray();
}
```

- **findEligibleAdvisors**方法：

```
protected List<Advisor> findEligibleAdvisors(Class<?> beanClass, String beanName) {
    // 获取所有可用的切面
    List<Advisor> candidateAdvisors = findCandidateAdvisors();
    List<Advisor> eligibleAdvisors = findAdvisorsThatCanApply(candidateAdvisors,
beanClass, beanName);
    extendAdvisors(eligibleAdvisors);
    if (!eligibleAdvisors.isEmpty()) {
        eligibleAdvisors = sortAdvisors(eligibleAdvisors);
    }
    return eligibleAdvisors;
}
```

- findAdvisorBeans获取所有可用的切面

- 从Bean工厂中获取所有Advisor（切面）的子类
- 根据名称从BeanFactory中获取切面的Bean实例
- 添加到advisors中去

```java
reAspectJAutoProxyCreator  AbstractAdvisorAutoProxyCreator.java  BeanFactoryAdvisorRetrievalHelper.java  BeanFactoryUtils.java  AbstractAutoProxyCreator
    */
    public List<Advisor> findAdvisorBeans() {
        // Determine list of advisor bean names, if not cached already.
        String[] advisorNames = null;
        synchronized (this) {
            advisorNames = this.cachedAdvisorBeanNames;
            if (advisorNames == null) {
                // Do not initialize FactoryBeans here: We need to leave all regular beans
                // uninitialized to let the auto-proxy creator apply to them!
                advisorNames = BeanFactoryUtils.beanNamesForTypeIncludingAncestors(
                        this.beanFactory, Advisor.class, includeNonSingletons: true, allowEagerInit: false);
                this.cachedAdvisorBeanNames = advisorNames;
            }
        }
        if (advisorNames.length == 0) {
            return new LinkedList<>();
        }

        List<Advisor> advisors = new LinkedList<>();
        for (String name : advisorNames) {
            if (isEligibleBean(name)) {
                if (this.beanFactory.isCurrentlyInCreation(name)) {
                    if (logger.isDebugEnabled()) {
                        logger.debug("Skipping currently created advisor '" + name + "'");
                    }
                }
                else {
                    try {
                        advisors.add(this.beanFactory.getBean(name, Advisor.class));
                    }
                    catch (BeanCreationException ex) {
                        Throwable rootCause = ex.getMostSpecificCause();
                        if (rootCause instanceof BeanCurrentlyInCreationException) {
                            BeanCreationException bce = (BeanCreationException) rootCause;
                            String bceBeanName = bce.getBeanName();
                            if (bceBeanName != null && this.beanFactory.isCurrentlyInCreation(bceBeanName)) {
                                if (logger.isDebugEnabled()) {
                                    logger.debug("Skipping advisor '" + name +
```

- 但是其实调用的子类AnnotationAwareAspectJAutoProxyCreator的findAdvisorBeans方法,从所有切面中获取可用的切面

```java
protected List<Advisor> findCandidateAdvisors() {
    List<Advisor> advisors = super.findCandidateAdvisors();
    if (this.aspectJAdvisorsBuilder != null) {
        advisors.addAll(this.aspectJAdvisorsBuilder.buildAspectJAdvisors());
    }
    return advisors;
}
```

- findAdvisorsThatCanApply方法调用

```java
public static List<Advisor> findAdvisorsThatCanApply(List<Advisor> candidateAdvisors,
Class<?> clazz) {
    // 如果切面为空，直接返回
```

```
    if (candidateAdvisors.isEmpty()) {
        return candidateAdvisors;
    }
    List<Advisor> eligibleAdvisors = new ArrayList<>();
    for (Advisor candidate : candidateAdvisors) {
        if (candidate instanceof IntroductionAdvisor && canApply(candidate, clazz)) {
            eligibleAdvisors.add(candidate);
        }
    }
    boolean hasIntroductions = !eligibleAdvisors.isEmpty();
    for (Advisor candidate : candidateAdvisors) {
        if (candidate instanceof IntroductionAdvisor) {
            // already processed
            continue;
        }
        if (canApply(candidate, clazz, hasIntroductions)) {
            eligibleAdvisors.add(candidate);
        }
    }
    return eligibleAdvisors;
}
```

- 先判断是否是引介切面（这个切面主要是针对于类，而不是方法，意思就是如果你引入这个切面，你做的增强可能是让目标类去实现一些接口，而不是给目标的的方法前后织入逻辑），后在判断put切面，但是最终都会进入canApply方法：

```
public static boolean canApply(Advisor advisor, Class<?> targetClass,
                               boolean hasIntroductions) {

    if (advisor instanceof IntroductionAdvisor) {
        return ((IntroductionAdvisor) advisor).getClassFilter().matches(targetClass);
    }
    else if (advisor instanceof PointcutAdvisor) {
        PointcutAdvisor pca = (PointcutAdvisor) advisor;
        return canApply(pca.getPointcut(), targetClass, hasIntroductions);
    }
    else {
        // It doesn't have a pointcut so we assume it applies.
        return true;
    }
}
```

- 总结：

  1.通过AspectJAutoProxyBeanDefinitionParser类将AnnotationAwareAspectJAutoProxyCreator注册到Spring容器中

  2.AnnotationAwareAspectJAutoProxyCreator类的postProcessAfterInitialization()方法将所有advice的bean包装成proxy

- AbstractAutoProxyCreator.wrapIfNecessary()分析中的createProxy()

所有的bean在返回给用户使用之前都需要经过AnnotationAwareAspectJAutoProxyCreator类的postProcessAfterInitialization()方法，而该方法的主要作用也就是将所有拥有advice的bean重新包装为proxy

```
 protected Object createProxy(
     Class<?> beanClass, String beanName, Object[] specificInterceptors, TargetSource
targetSource) {

     if (this.beanFactory instanceof ConfigurableListableBeanFactory) {
         AutoProxyUtils.exposeTargetClass((ConfigurableListableBeanFactory)
this.beanFactory, beanName, beanClass);
     }

     // 1.创建proxyFactory, proxy的生产主要就是在proxyFactory做的
     ProxyFactory proxyFactory = new ProxyFactory();
     proxyFactory.copyFrom(this);

     if (!proxyFactory.isProxyTargetClass()) {
         if (shouldProxyTargetClass(beanClass, beanName)) {
             proxyFactory.setProxyTargetClass(true);
         }
         else {
             evaluateProxyInterfaces(beanClass, proxyFactory);
         }
     }

     // 2.将当前bean适合的advice, 重新封装下, 封装为Advisor类, 然后添加到ProxyFactory中
     Advisor[] advisors = buildAdvisors(beanName, specificInterceptors);
     for (Advisor advisor : advisors) {
         proxyFactory.addAdvisor(advisor);
     }

     proxyFactory.setTargetSource(targetSource);
     customizeProxyFactory(proxyFactory);

     proxyFactory.setFrozen(this.freezeProxy);
     if (advisorsPreFiltered()) {
         proxyFactory.setPreFiltered(true);
     }

     // 3.调用getProxy获取bean对应的proxy
     return proxyFactory.getProxy(getProxyClassLoader());
 }
```

- 创建何种类型的Proxy? JDKProxy还是CGLIBProxy?

```
 // getProxy()方法
public Object getProxy(ClassLoader classLoader) {
    return createAopProxy().getProxy(classLoader);
}
```

// createAopProxy()方法就是决定究竟创建何种类型的proxy protected final synchronized AopProxy createAopProxy() {

```
  if (!this.active) {
      activate();
  }
  // 关键方法createAopProxy()
  return getAopProxyFactory().createAopProxy(this);
```

}

// createAopProxy() public AopProxy createAopProxy(AdvisedSupport config) throws AopConfigException {

```
  // 1.config.isOptimize()是否使用优化的代理策略，目前使用与CGLIB
  // config.isProxyTargetClass() 是否目标类本身被代理而不是目标类的接口
  // hasNoUserSuppliedProxyInterfaces()是否存在代理接口
  if (config.isOptimize() || config.isProxyTargetClass() ||
hasNoUserSuppliedProxyInterfaces(config)) {
      Class<?> targetClass = config.getTargetClass();
      if (targetClass == null) {
          throw new AopConfigException("TargetSource cannot determine target class: " +
                                       "Either an interface or a target is required for
proxy creation.");
      }

      // 2.如果目标类是接口或者是代理类，则直接使用JDKproxy
      if (targetClass.isInterface() || Proxy.isProxyClass(targetClass)) {
          return new JdkDynamicAopProxy(config);
      }

      // 3.其他情况则使用CGLIBproxy
      return new ObjenesisCglibAopProxy(config);
  }
  else {
      return new JdkDynamicAopProxy(config);
  }
```

}

```
- getProxy()方法,可知，通过createAopProxy()方法来确定具体使用何种类型的Proxy

  ```java

  final class JdkDynamicAopProxy implements AopProxy, InvocationHandler, Serializable//
JdkDynamicAopProxy类结构，由此可知，其实现了InvocationHandler，则必定有invoke方法，来被调用，也就是
用户调用bean相关方法时，此invoke()被真正调用

      // getProxy()
      public Object getProxy(ClassLoader classLoader) {

        if (logger.isDebugEnabled()) {
```
```

```
            logger.debug("Creating JDK dynamic proxy: target source is " +
this.advised.getTargetSource());
        }
        Class<?>[] proxiedInterfaces =
AopProxyUtils.completeProxiedInterfaces(this.advised, true);
        findDefinedEqualsAndHashCodeMethods(proxiedInterfaces);

          // JDK proxy 动态代理的标准用法
        return Proxy.newProxyInstance(classLoader, proxiedInterfaces, this);

  }
```

- invoke()方法， 使用了JDK动态代理模式，真正的方法执行在invoke()方法里

```
public Object invoke(Object proxy, Method method, Object[] args) throws Throwable {
        MethodInvocation invocation;
        Object oldProxy = null;
        boolean setProxyContext = false;

        TargetSource targetSource = this.advised.targetSource;
        Class<?> targetClass = null;
        Object target = null;

        try {
            // 1.以下的几个判断，主要是为了判断method是否为equals、hashCode等Object的方法
            if (!this.equalsDefined && AopUtils.isEqualsMethod(method)) {
                return equals(args[0]);
            }
            else if (!this.hashCodeDefined && AopUtils.isHashCodeMethod(method)) {
                return hashCode();
            }
            else if (method.getDeclaringClass() == DecoratingProxy.class) {
                return AopProxyUtils.ultimateTargetClass(this.advised);
            }
            else if (!this.advised.opaque && method.getDeclaringClass().isInterface()
&&
                    method.getDeclaringClass().isAssignableFrom(Advised.class)) {
                return AopUtils.invokeJoinpointUsingReflection(this.advised, method,
args);
            }

            Object retVal;

            if (this.advised.exposeProxy) {
                oldProxy = AopContext.setCurrentProxy(proxy);
                setProxyContext = true;
            }

            target = targetSource.getTarget();
            if (target != null) {
                targetClass = target.getClass();
            }
```

```
            // 2.获取当前bean被拦截方法链表
            List<Object> chain =
this.advised.getInterceptorsAndDynamicInterceptionAdvice(method, targetClass);

            // 3.如果为空，则直接调用target的method
            if (chain.isEmpty()) {
                Object[] argsToUse = AopProxyUtils.adaptArgumentsIfNecessary(method,
args);
                retVal = AopUtils.invokeJoinpointUsingReflection(target, method,
argsToUse);
            }
            // 4.不为空，则逐一调用chain中的每一个拦截方法的proceed
            else {
                invocation = new ReflectiveMethodInvocation(proxy, target, method,
args, targetClass, chain);
                retVal = invocation.proceed();
            }

            ...
            return retVal;
        }
        ...
    }
```

- 拦截方法真正被执行调用invocation.proceed()

```
public Object proceed() throws Throwable {
    if (this.currentInterceptorIndex ==
this.interceptorsAndDynamicMethodMatchers.size() - 1) {
        return invokeJoinpoint();
    }

    Object interceptorOrInterceptionAdvice =

 this.interceptorsAndDynamicMethodMatchers.get(++this.currentInterceptorIndex);
    if (interceptorOrInterceptionAdvice instanceof
InterceptorAndDynamicMethodMatcher) {
        InterceptorAndDynamicMethodMatcher dm =
            (InterceptorAndDynamicMethodMatcher) interceptorOrInterceptionAdvice;
        if (dm.methodMatcher.matches(this.method, this.targetClass, this.arguments))
{
            return dm.interceptor.invoke(this);
        }
        else {
            return proceed();
        }
    }
    else {
        return ((MethodInterceptor) interceptorOrInterceptionAdvice).invoke(this);
    }
}
```

总结：依次遍历拦截器链的每个元素，然后调用其实现，将真正调用工作委托给各个增强器

总结：

纵观以上过程可知：实际就是为bean创建一个proxy，JDKproxy或者CGLIBproxy，然后在调用bean的方法时，会通过proxy来调用bean方法

重点过程可分为：

1）通过AspectJAutoProxyBeanDefinitionParser类将AnnotationAwareAspectJAutoProxyCreator注册到Spring容器中

2）AnnotationAwareAspectJAutoProxyCreator类的postProcessAfterInitialization()方法将所有有advice的bean重新包装成proxy

3）调用bean方法时通过proxy来调用，proxy依次调用增强器的相关方法，来实现方法切入